



COSI DẠY CODE

Assignment 1: Calculator

Người Soạn: Phương Xương Đức
Ho Chi Minh City, *April, 2026*



Mục lục

1	Thiết kế chi tiết lớp Fraction (Phân số)	2
1.1	Cấu trúc dữ liệu	2
1.2	Các phương thức quan trọng	2
1.3	Nạp chồng toán tử (Operator Overloading)	2
2	Thiết kế chi tiết lớp AccuracyFloat	4
2.1	Cấu trúc dữ liệu	4
2.2	Các hàm khởi tạo và chuyển đổi	4
2.3	Nạp chồng toán tử (Operator Overloading)	4

1 Thiết kế chi tiết lớp Fraction (Phân số)

Lớp Fraction được sử dụng để biểu diễn các số dưới dạng tỉ số giữa hai số nguyên. Đây là giải pháp tối ưu để xử lý các số thập phân vô hạn tuần hoàn mà không làm mất đi độ chính xác của dữ liệu.

1.1 Cấu trúc dữ liệu

- **numerator (long long):** Lưu trữ tử số (giá trị tuyệt đối).
- **denominator (long long):** Lưu trữ mẫu số (luôn dương và khác 0).
- **isNegative (bool):** Cờ hiệu xác định dấu của phân số.

1.2 Các phương thức quan trọng

- **Fraction(long long num, long long den, bool sign):** Hàm khởi tạo với tử số, mẫu số và dấu.
- **Fraction(string input):** Khởi tạo từ dạng chuỗi (vd: "-1/3").
- **simplify():** Hàm rút gọn phân số bằng cách chia cả tử và mẫu cho Ước chung lớn nhất (GCD).
- **toString(): string:** Trả về chuỗi định dạng "tử/mẫu" (ví dụ: "-3/4").
- **toAccuracyFloat():** Phương thức hỗ trợ chuyển đổi phân số sang số thực chính xác cao khi cần thiết.

1.3 Nạp chồng toán tử (Operator Overloading)

Việc nạp chồng toán tử giúp thực hiện các phép tính hằng ngày trên phân số một cách tự nhiên nhất[cite: 1]:

1. Toán tử số học:

- **operator+(Fraction other): Fraction:** Cộng hai phân số (cần quy đồng mẫu số).
- **operator-(Fraction other): Fraction:** Trừ hai phân số.
- **operator*(Fraction other): Fraction:** Nhân tử với tử, mẫu với mẫu.
- **operator/(Fraction other): Fraction:** Nhân nghịch đảo phân số.

2. Toán tử so sánh:

- **operator==(Fraction other): bool**
- **operator!=(Fraction other): bool**

- `operator>(Fraction other): bool`
- `operator<(Fraction other): bool`

Fraction
- numerator: long long - denominator: long long - isNegative: bool
+ Fraction(long long num, long long den, bool sign) + simplify(): void + toString(): string + toAccuracyFloat(): AccuracyFloat + operator+(other: Fraction): Fraction + operator-(other: Fraction): Fraction + operator*(other: Fraction): Fraction + operator/(other: Fraction): Fraction + operator==(other: Fraction): bool + operator!=(other: Fraction): bool + operator>(other: Fraction): bool + operator<(other: Fraction): bool

Figure 1: Sơ đồ cấu trúc lớp Fraction

2 Thiết kế chi tiết lớp AccuracyFloat

Lớp AccuracyFloat được thiết kế để xử lý các số thực với độ chính xác cao bằng cách chia tách phần nguyên và phần thập phân, kết hợp với quản lý dấu riêng biệt.

2.1 Cấu trúc dữ liệu

- **integerPart (int):** Lưu trữ giá trị tuyệt đối của phần nguyên.
- **decimalPart (long long):** Lưu trữ giá trị phần thập phân dưới dạng số nguyên lớn.
- **precisePart (long long):** Xác định số lượng chữ số sau dấu phẩy (quyết định vị trí dấu chấm thập phân).
- **isNegative (bool):** Lưu trữ dấu của số (True nếu âm, False nếu dương).

2.2 Các hàm khởi tạo và chuyển đổi

- **AccuracyFloat(int integerPart, long long decimalPart, long long precisePart, bool isNegative):** Khởi tạo đầy đủ các thành phần.
- **AccuracyFloat(double input):** Chuyển đổi từ kiểu dữ liệu double (có thể gặp sai số).
- **AccuracyFloat(string input):** Khởi tạo từ chuỗi văn bản (đảm bảo độ chính xác tuyệt đối).
- **toString(): string:** Trả về chuỗi đại diện cho số ở định dạng thập phân.
- **toDouble(): double:** Chuyển đổi ngược lại kiểu double khi cần tính toán nhanh.

2.3 Nạp chồng toán tử (Operator Overloading)

Các toán tử được nạp chồng để thực hiện tính toán trực tiếp trên đối tượng AccuracyFloat:

1. Toán tử số học:

- **operator+(AccuracyFloat other):** AccuracyFloat
- **operator-(AccuracyFloat other):** AccuracyFloat
- **operator*(AccuracyFloat other):** AccuracyFloat
- **operator/(AccuracyFloat other):** AccuracyFloat

2. Toán tử so sánh:

- `operator==(AccuracyFloat other): bool`
- `operator!=(AccuracyFloat other): bool`
- `operator>(AccuracyFloat other): bool`
- `operator<(AccuracyFloat other): bool`
- `operator>=(AccuracyFloat other): bool`
- `operator<=(AccuracyFloat other): bool`

AccuracyFloat
- integerPart: long long - decimalPart: long long - precisePart: long long - isNegative: bool
+ AccuracyFloat(int integerPart, long long decimalPart, long long precisePart, bool isNegative) + AccuracyFloat(double input) + AccuracyFloat(string input) + toString(): string + toDouble(): double
+ operator+(AccuracyFloat other): AccuracyFloat + operator-(AccuracyFloat other): AccuracyFloat + operator*(AccuracyFloat other): AccuracyFloat + operator/(AccuracyFloat other): AccuracyFloat
+ operator==(AccuracyFloat other): bool + operator!=(AccuracyFloat other): bool + operator>(AccuracyFloat other): bool + operator<(AccuracyFloat other): bool + operator>=(AccuracyFloat other): bool + operator<=(AccuracyFloat other): bool

Figure 2: Sơ đồ cấu trúc lớp AccuracyFloat